Prof. Moreira - Unisalesiano - Dispositivos Móveis 1

Multimídia & Base64 no Android

Gravação de áudio com MediaRecorder, reprodução com MediaPlayer e codificação Base64 para envio via API em segundo plano.

Estudo de Caso Prático: App Controle de Equipamentos / Chamado Técnico

CONFIGURAÇÃO INICIAL DO SISTEMA

Permissão no AndroidManifest.xml e Arquitetura do App

1. DECLARAÇÃO OBRIGATÓRIA NO ARQUIVO AndroidManifest.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.chamadotecnico">

    <!-- Permissão necessária para capturar áudio pelo microfone -->
    <uses-permission android:name="android.permission.RECORD_AUDIO"/>

    <application ...>
```

2. FLUXO GERAL DE FUNCIONAMENTO DO APLICATIVO DE SUPORTE TÉCNICO:

- Microfone & MediaRecorder: Grava o relato de voz do operador no cache local (/audio_falha.m4a).
- Conversão para Base64: Converte o arquivo de áudio binário em String texto via Base64.encodeToString().
- Transmissão Assíncrona: Conecta à API em uma Thread secundária e envia formulário + áudio em UTF-8.
- Feedback ao Usuário: Exibe Toast e zera os campos na UI principal usando runOnUiThread().

SEGURANÇA NO ANDROID 6.0+

Solicitação de Permissão Dinâmica em Tempo de Execução

MÉTODOS DE PERMISSÃO (JAVA)

```
// 1. Verificar se a permissão de microfone já foi concedida
if (ContextCompat.checkSelfPermission(this, Manifest.permission.RECORD_AUDIO)
        != PackageManager.PERMISSION_GRANTED) {

    // 2. Solicitar permissão dinamicamente ao usuário
    ActivityCompat.requestPermissions(
        this,
        new String[]{Manifest.permission.RECORD_AUDIO},
        REQUEST_RECORD_AUDIO_PERMISSION
    );
}
```

POR QUE SOLICITAR EM TEMPO DE EXECUÇÃO?

- **Permissões Perigosas:**
  O microfone é classificado como dado sensível de privacidade.

- **Check Dinâmico:**
  A tag no Manifest não basta sozinha em versões Android 6.0 (API 23) ou superior.

- **Requisito da API:**
  O método checkSelfPermission() evita a exceção SecurityException ao tentar gravar sem autorização.

API MULTIMÍDIA ANDROID

Captura e Gravação de Áudio com MediaRecorder

INICIALIZAÇÃO DO MEDIARECORDER (JAVA)

```
private void iniciarGravacaoAudio() {
    mediaRecorder = new MediaRecorder();
    mediaRecorder.setAudioSource(MediaRecorder.AudioSource.MIC);
    mediaRecorder.setOutputFormat(MediaRecorder.OutputFormat.MPEG_4);
    mediaRecorder.setAudioEncoder(MediaRecorder.AudioEncoder.AAC);
    mediaRecorder.setOutputFile(caminhoAudio);

    try {
        mediaRecorder.prepare();
        mediaRecorder.start();
        isRecording = true;
    } catch (IOException e) {
        Log.e("AUDIO", "Falha ao preparar gravação", e);
    }
}
```

CONFIGURAÇÃO ETAPA POR ETAPA

1. setAudioSource(MIC):
Define o microfone físico como fonte de captura.

2. setOutputFormat(MPEG_4):
Define o container do arquivo de saída (.m4a).

3. setAudioEncoder(AAC):
Define o codec de compressão de áudio.

4. prepare() & start():
Aloca recursos de hardware e inicia a gravação.

API MULTIMÍDIA ANDROID

Reprodução do Áudio Gravado com MediaPlayer

MÉTODO REPRODUZIRAUDIOGRAVADO() (JAVA)

```
private void reproduzirAudioGravado() {
    File file = new File(caminhoAudio);
    if (!file.exists()) return;

    if (mediaPlayer != null) {
        mediaPlayer.release();
    }

    mediaPlayer = new MediaPlayer();
    try {
        mediaPlayer.setDataSource(caminhoAudio);
        mediaPlayer.prepare();
        mediaPlayer.start();
    } catch (IOException e) {
        Log.e("AUDIO", "Erro ao reproduzir", e);
    }
}
```

CICLO DO MEDIAPLAYER

- **Liberação Prévia:**
  Invocamos mediaPlayer.release() para desalocar a memória de execuções anteriores.

- **Origem dos Dados:**
  setDataSource() recebe o caminho do arquivo no cache local (/audio_falha.m4a).

- **Buffering & Play:**
  prepare() carrega os áudios e start() inicia a emissão no alto-falante do aparelho.

SERIALIZAÇÃO DE DADOS MULTIMÍDIA

Conversão de Arquivo Binário em Texto (Base64)

CONVERSÃO PARA BASE64 (JAVA)

```
private String converterAudioParaBase64(String path) {
    File file = new File(path);
    if (!file.exists()) return "";

    byte[] bytes = new byte[(int) file.length()];
    try (FileInputStream fis = new FileInputStream(file)) {
        fis.read(bytes);
        // Transforma o array de bytes em String Base64
        return Base64.encodeToString(bytes, Base64.NO_WRAP);
    } catch (IOException e) {
        e.printStackTrace();
        return "";
    }
}
```

O CONCEITO DE BASE64

- **O que é Base64?**
  Algoritmo de codificação que transforma dados binários em caracteres ASCII seguros.

- **Vantagem na Web:**
  Permite trafegar arquivos (som, imagem) dentro de corpos JSON ou parâmetros de URL HTTP.

- **Flag NO_WRAP:**
  Impede a inserção automática de quebras de linha (\n) que corromperiam a requisição.

COMUNICAÇÃO DE REDE ASSÍNCRONA

Transmissão HTTP e Atualização de Interface

THREAD & RUNONUITHREAD (JAVA)

```
new Thread(() -> {
    try {
        String audioBase64 = converterAudioParaBase64(caminhoAudio);
        String params = "funcionario=" + URLEncoder.encode(func, "UTF-8") +
                        "&audio=" + URLEncoder.encode(audioBase64, "UTF-8");

        // Conexão HTTP POST via HttpURLConnection...

        runOnUiThread(() -> {
            Toast.makeText(this, "Chamado Enviado!", Toast.LENGTH_LONG).show();
            limparFormulario();
        });
    } catch (Exception e) { ... }
}).start();
```

REGRAS DE REDE NO ANDROID

- **Thread Secundária:**
  O Android proíbe conexões de rede na MainThread (evita congelamento da tela).

- **URLEncoder.encode():**
  Converte caracteres especiais da String Base64 para formato legível em URL.

- **runOnUiThread():**
  Apenas a MainThread pode alterar componentes da interface (ex: Toast, TextView).

MATRIZ DE CONSOLIDAÇÃO

Resumo dos Conceitos Práticos Aplicados

1. Permissões de Hardware

Declaração estática no AndroidManifest.xml (<uses-permission android:name="android.permission.RECORD_AUDIO"/>) e validação dinâmica com ActivityCompat em tempo de execução.

2. Captura com MediaRecorder

Configuração da fonte (MIC), formato de saída (MPEG_4) e codificador (AAC). Ciclo com prepare(), start() e stop().

3. Reprodução com MediaPlayer

Gerenciamento da fonte de áudio local, controle de memória com release() e emissão de som com prepare() e start().

4. Base64 & Comunicação Assíncrona

Conversão de bytes do arquivo .m4a em String para envio em requisição HTTP usando Thread em segundo plano e atualização na UI.